

Applied Computer Vision and Deep Learning

UBCO Master of Data Science – DATA 589





Deep Learning

- What is it?
Extract useful patterns and knowledge from data/image
- How?
Neural Network + Optimization
- How?
Python + TensorFlow + PyTorch
- Hard Part!
Good data + applying methodologies to real world problems
- Why now?
Data, hardware, community, tools, investment
- Where are we now?
LLMs, VLMs, GenAI, AI Agents

CNNs History

Hubel & Wiesel (1959)

Discovery of simple and complex cells in the cat visual cortex, revealing how neurons respond to specific orientations and patterns.

Hierarchical Processing

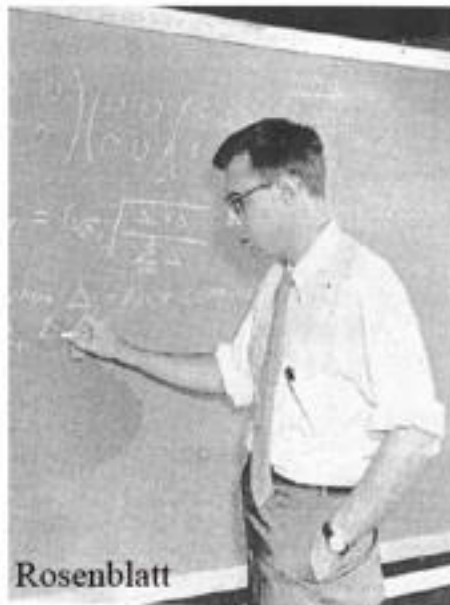
Vision starts with simple edges and builds up to complex shapes and objects through successive layers of abstraction.

Local Connectivity

Neurons respond to specific receptive fields, a core principle that defines the convolutional layers in modern CNNs.

When Did the Story Start?

Perceptrons, 1958



http://www.ccie.rpi.edu/homepages/nagy/PDF_files/2011_Nagy_Perce_FE.pdf Photo by George Nagy



<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.335.3398&rep=rep1&type=pdf>

Deep Learning Milestones – CV Focused

- **1943:** Formal neuron model establishes the foundation of neural networks.
- **1958:** The perceptron introduces learnable linear classifiers.
- **1986:** Backpropagation enables practical training of multi-layer neural networks.
- **1989–1998:** CNN mature through LeNet-style models for visual recognition.
- **2006:** The term “deep learning” gains traction with deep belief networks and unsupervised pretraining.
- **2009:** ImageNet marks the beginning of large-scale data driving model performance.

Deep Learning Milestones – CV Focused

- 2012: AlexNet demonstrates the power of deep CNNs trained with GPUs, reshaping computer vision.
- 2014: Generative Adversarial Networks introduce modern deep generative modeling.
- 2015: Residual networks enable very deep architectures and become the dominant CV backbone.
- 2017: Transformers establish attention as a core architectural principle.
- 2018: BERT popularizes large-scale pretraining and transfer learning.
- 2019: EfficientNet formalizes principled scaling of neural networks.
- 2020: Vision Transformers and DETR bring transformer architectures into mainstream computer vision.
- 2020: Diffusion models emerge as a powerful paradigm for image generation.
- 2021: Vision–language models enable zero-shot and open-vocabulary visual understanding.
- 2022: ChatGPT demonstrates instruction-aligned foundation models and changes how AI systems are used.
- 2022: Latent diffusion models make high-quality image generation efficient and widely accessible.
- 2023: Promptable foundation vision models redefine segmentation and interactive perception.
- 2024: Real-time multimodal models integrate vision, language, and audio into unified systems.
- 2024–2025: Text-to-video and world-model approaches signal a shift toward

What Comes Next?

Deep learning is evolving from predictive models to interactive, autonomous systems.

- AI Agents – systems that perceive, reason, plan, use tools, and act over long time horizons
- World Models – learned internal representations of the environment for prediction, simulation, and planning
- Embodied AI – grounding perception and learning through physical interaction (robots, drones, vehicles)
- Multimodal Foundation Systems – unified vision, language, audio, and action model
- Simulation + Synthetic Data – scalable training through digital twins and world simulators
- Continual & Lifelong Learning – adapting models after deployment without catastrophic forgetting
- Data-Centric & Trustworthy AI – robustness, uncertainty, alignment, and reliability as first-class goals

What is Deep Learning Today?

Deep learning today is best understood as a scalable systems paradigm, not just a model.

- Automatic differentiation frameworks (e.g., PyTorch, TensorFlow) automatically compute gradients for training
- Extremely large datasets, ranging from millions to billions of data points
- Distributed and parallel training across hundreds to thousands of GPUs/TPUs
- Very large neural architectures with billions (and beyond) parameters
- Training costs that can reach millions of dollars for frontier models
- Performance that often exceeds human-level benchmarks on narrow tasks
- Scale helps, but is not always necessary (e.g., compact diffusion and fine-tuned models such as Stable Diffusion)
- A strong open-source ecosystem enabling rapid iteration, reuse, and modular composition

AlexNet

AlexNet proved that scale, compute, and algorithms are the keys to visual intelligence.

The 2012 ImageNet competition marked the "Big Bang" of deep learning, where AlexNet nearly halved the error rate of traditional methods.

Big Data

ImageNet provided a massive dataset of 1.4 million labeled images across 1,000 classes, offering the necessary scale for deep networks to generalize.

GPU Compute

The shift to **NVIDIA GPUs** allowed for the massive parallelization of matrix operations, making the training of deep CNNs computationally feasible.

Deep Algorithms

The use of **ReLU activation**, Dropout for regularization, and deep convolutional layers enabled the training of much larger and more capable models.

Question

Question: Which concept introduced the idea that visual processing occurs hierarchically from simple to complex patterns?

- A. AlexNet
- B. Hubel & Wiesel's visual cortex experiments
- C. ImageNet dataset
- D. Transformers
- E. All the above



Deep Learning

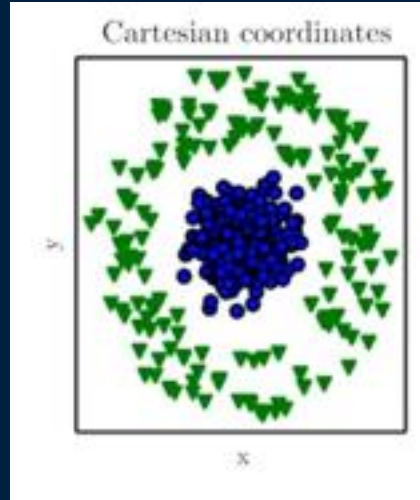


Deep Learning = Representation Learning

Deep learning is fundamentally a form of **representation learning** (also known as **feature learning**). While traditional machine learning relies on "hand-crafted" features designed by human experts, deep learning models automatically discover the representations needed for tasks like **classification** or **detection** directly from **raw data**.

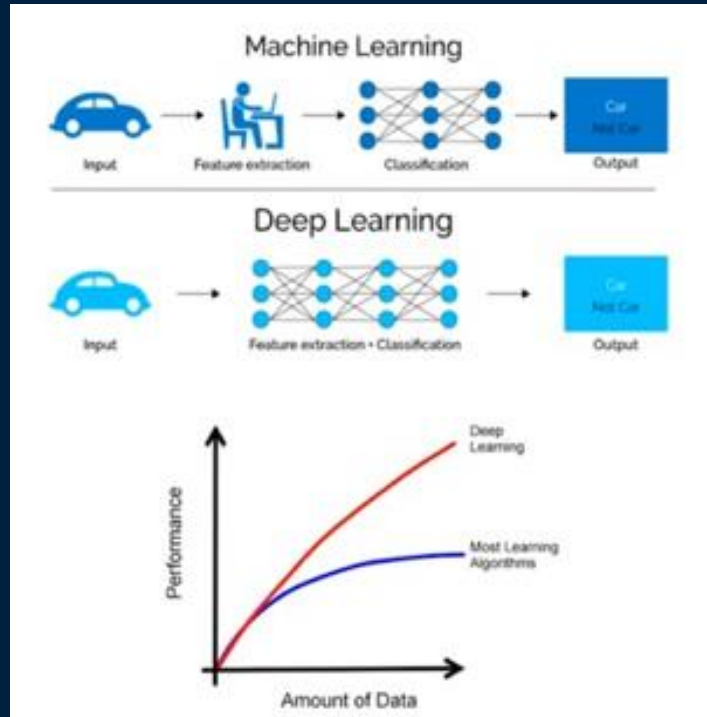
Why Representations Matter

Example: how to draw a line to separate the green triangles and blue circles?



Lex Fridman - <https://deeplearning.mit.edu/>

Deep Learning is Scalable



Lex Fridman - <https://deeplearning.mit.edu/>

Feature Extraction - Classification

High Level Feature Detection

Let's identify key features in each image category



Nose,
Eyes,
Mouth



Wheels,
License Plate,
Headlights

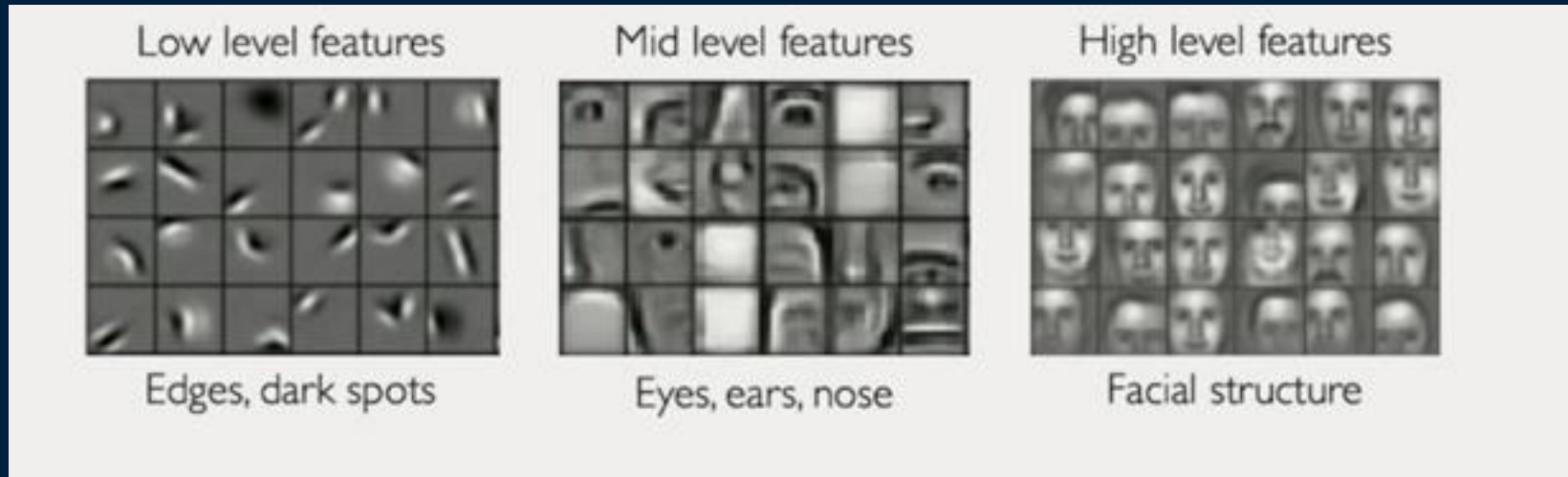


Door,
Windows,
Steps



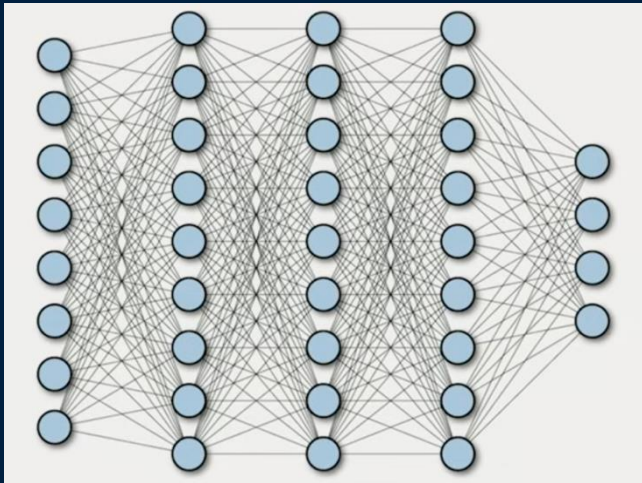
Learning Feature Representations

Can we learn a **hierarchy of features** directly from the **data** instead of hand engineering? Yes, but it needs **big data**!



MIT: <https://deeplearning.mit.edu/>

Learning Visual Features



Fully Connected Network - FCN

Deep neural networks learn hierarchical feature representations implicitly, as opposed to explicitly defined, hand-crafted features.

Question

Question: What major event triggered the modern deep learning revolution in computer vision?

- A. Development of PyTorch
- B. Release of the COCO dataset
- C. AlexNet winning the ImageNet challenge in 2012
- D. Introduction of backpropagation for multi-layer neural networks
- E. The invention of the perceptron

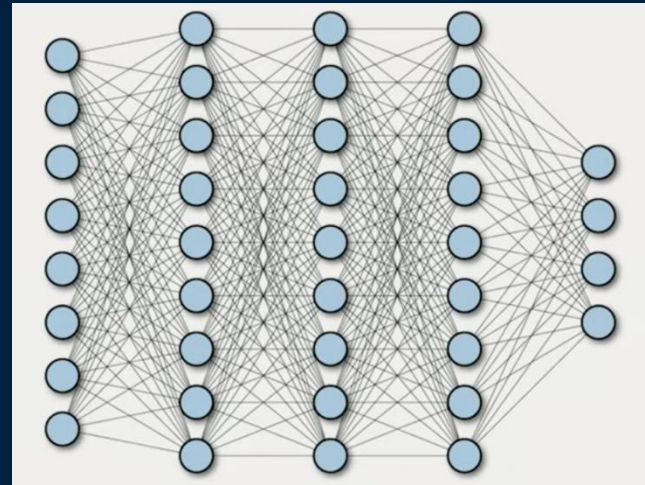
Convolutional Neural Network

Fully Connected Networks (FCNs)

A Fully Connected Network is a neural network where **every neuron in one layer is connected to every neuron in the next layer**.

- Input data is converted into a **1D vector**
- Each layer computes a weighted sum plus bias
- Final layer produces the prediction

» $y=f(Wx+b)$



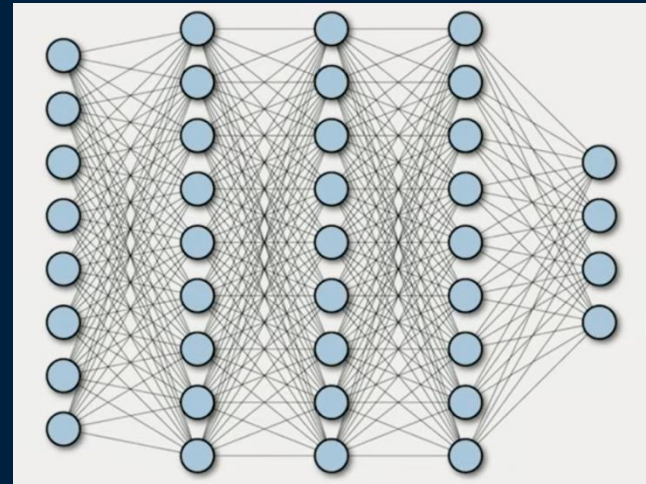
Fully Connected Neural Network - FCN

FCN is not great for image data!

When input is a 2D image:

The image needs to be flattened to a 1-D vector

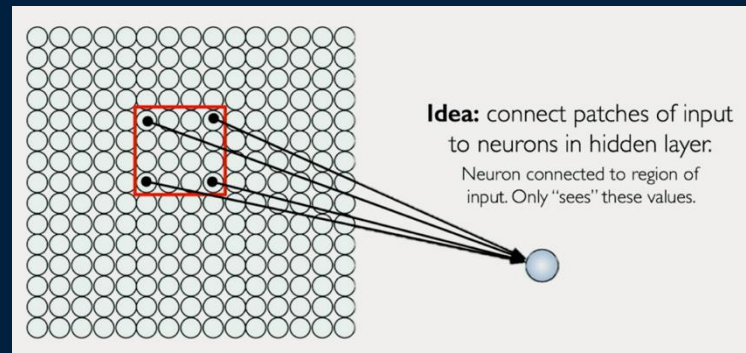
- No spatial information
- Too many parameters



How to Preserve Spatial Structure?

One way to keep **Spatial information** is to connect **patches of image** to each neuron!

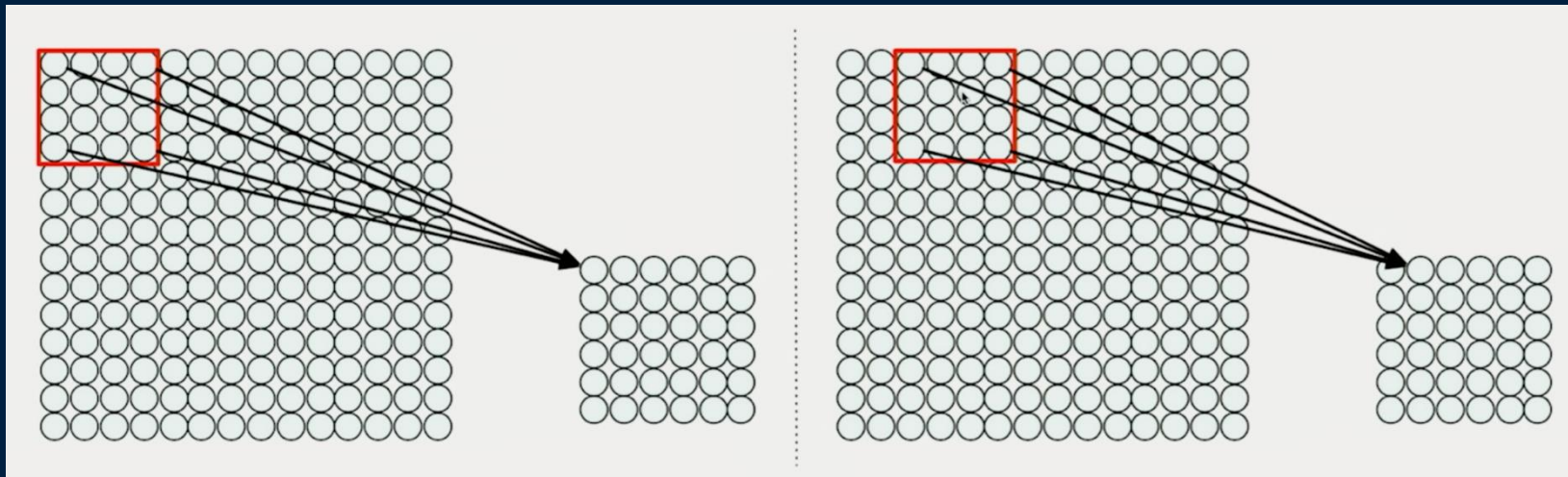
Connect each input pixels to **local output neuron**. For example, in this image the neuron will only see the inputs from the red patch on the left.



Spatial Structure

Connect **patch** in input layer to a single neuron in subsequent layer.
Use a **sliding window** to define connections.

But how can we weight the patch to detect features?



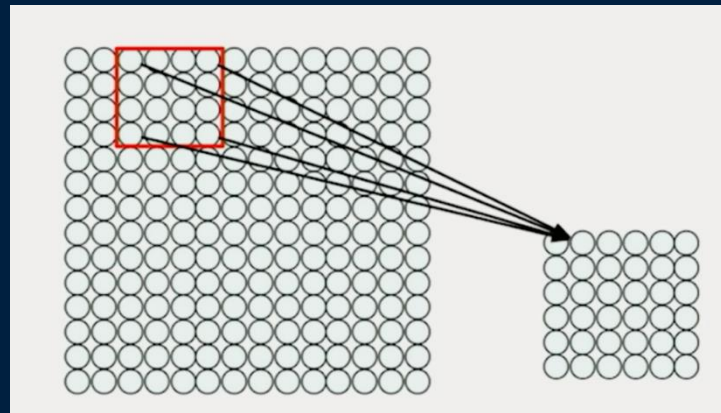


Feature Extraction with Convolution

- In practice the **red box** is called a **filter**!
- Filter of size $4 \times 4 = 16$ different weights
- Apply the same filter to all 4×4 patches in input image
- Shift by two pixels for next patch – **sliding window**

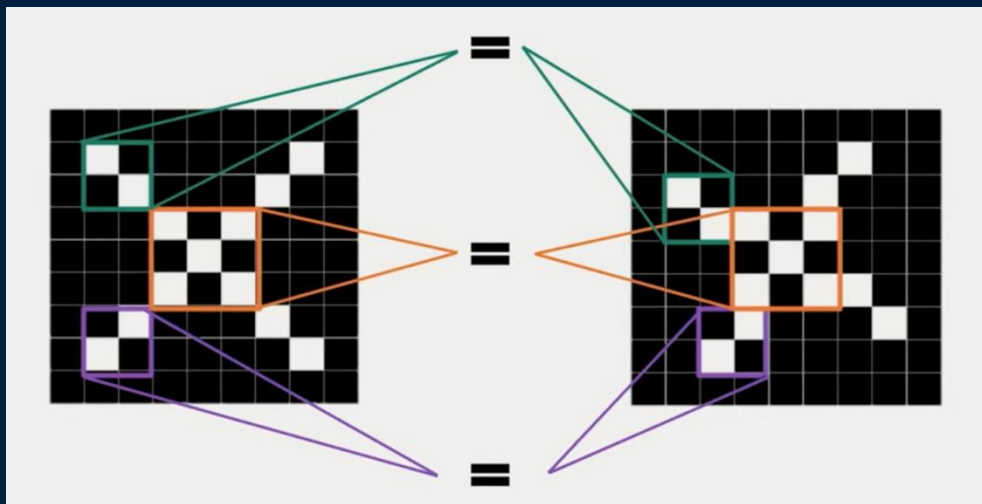
This patchy operation is called **Convolution**!

1. Apply a set of weights – a filter – to extract local features
2. Use multiple filters to extract different features

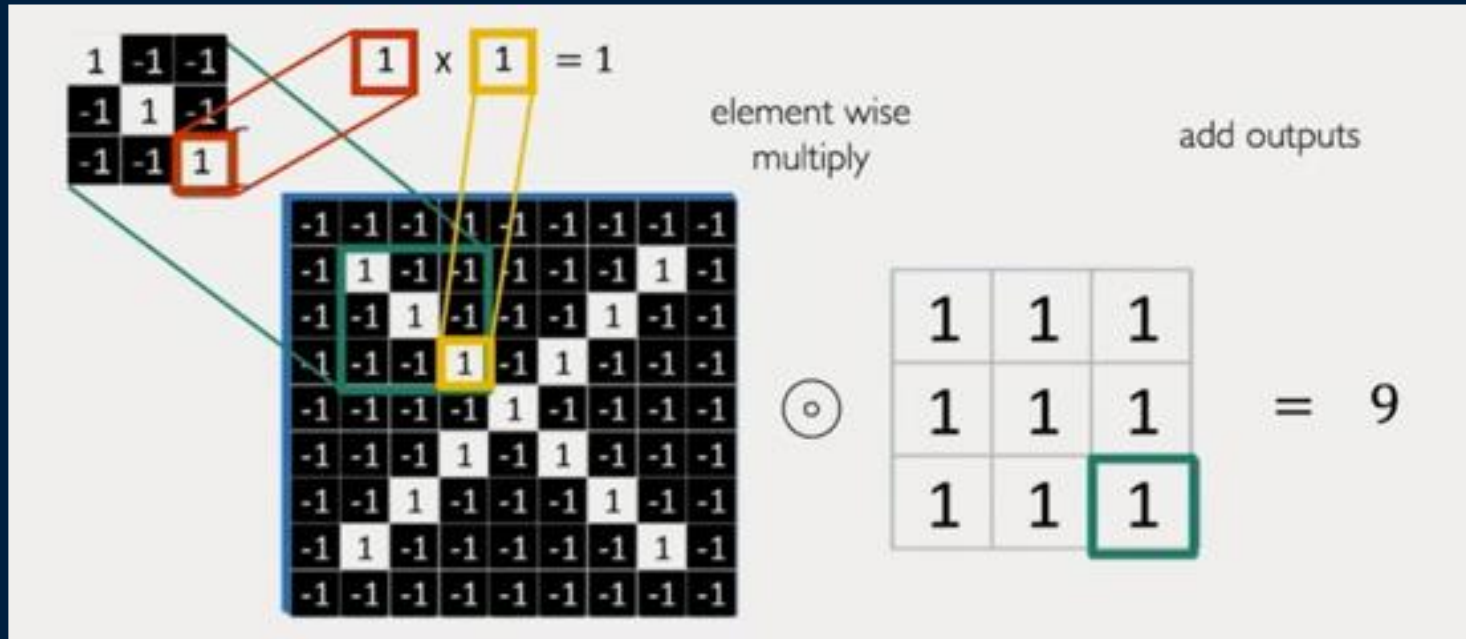


Feature Extraction and Convolution

- Are the two different pictures the same? A **Classification problem!**
- If there are **enough the of the same patterns** then we can classify them as one class.



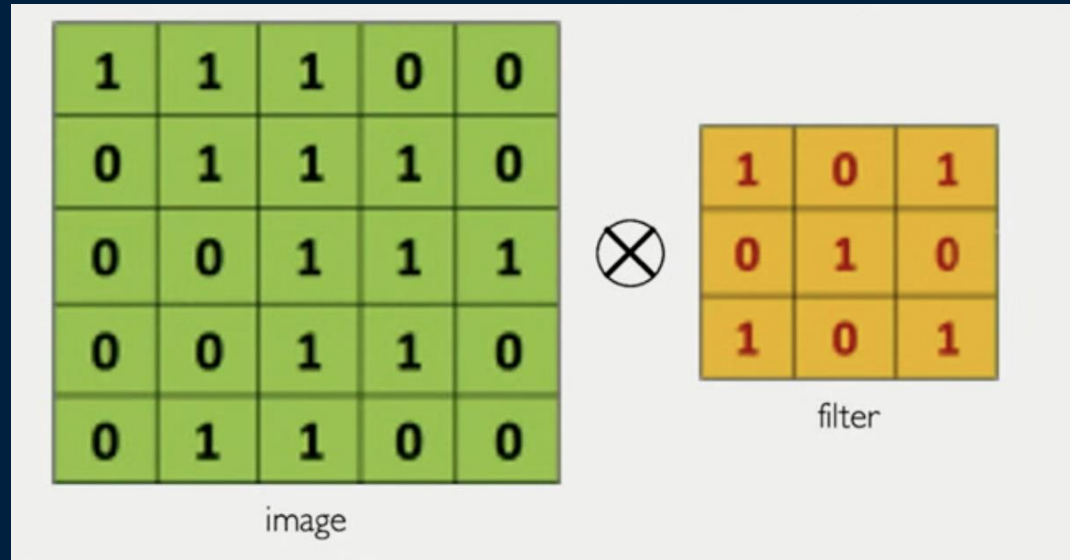
Convolution Operation



Convolution Operation

Another example!

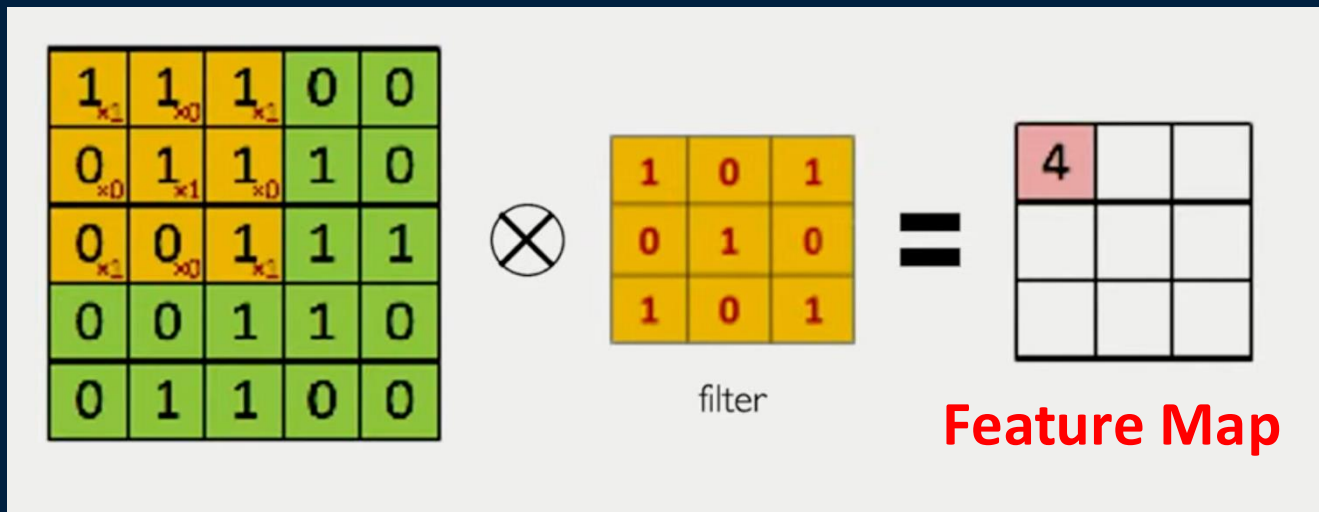
5x5 image with a kernel of 3x3 how does it work?



Convolution Operation

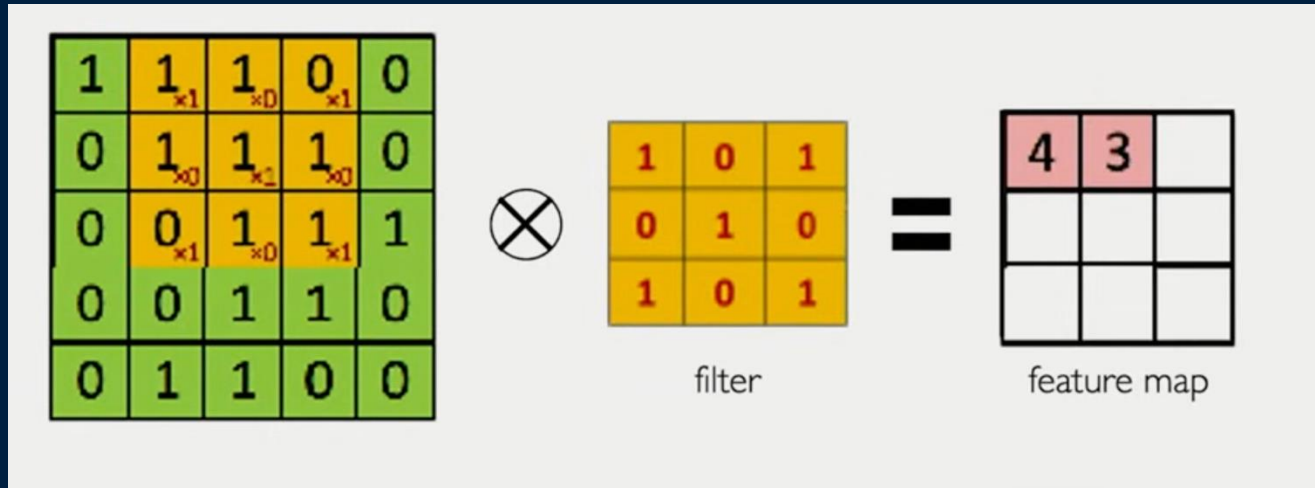
Another example!

5x5 image with a kernel of 3x3 how does it work?



Convolution Operation

Output array called **Feature map**



Convolution Operation

Note: If the input image has more than one dimension like RGB then the filters would have more than one dimension like 3 for RGB image. But the **feature map would have one dimension** because for each patch when applying the element-wise multiply the result will be sum of all channels for that pixel.

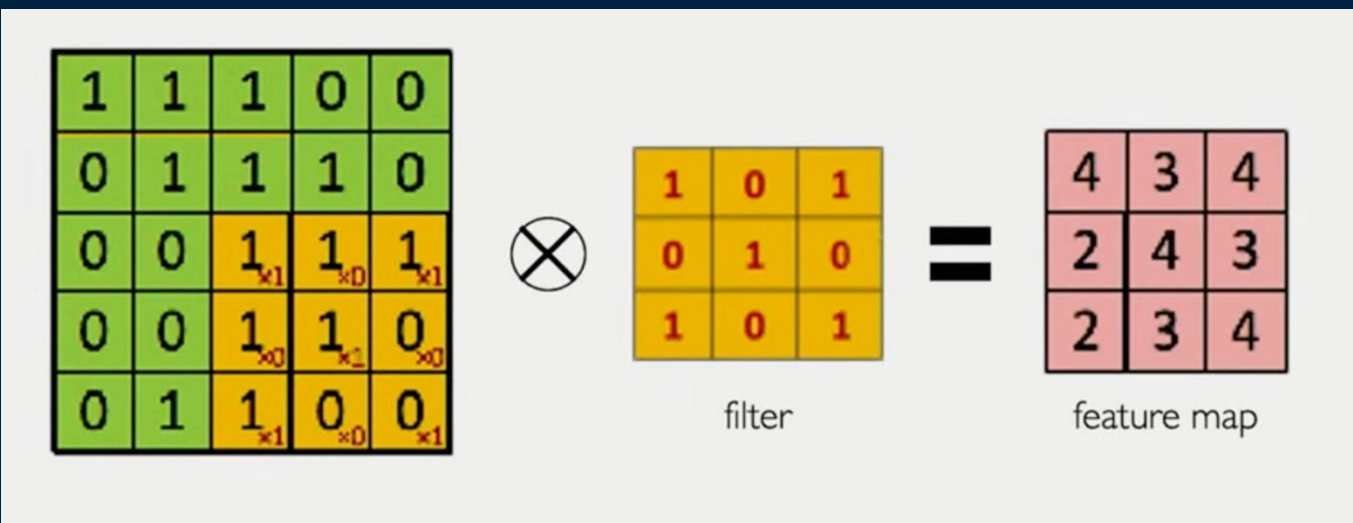
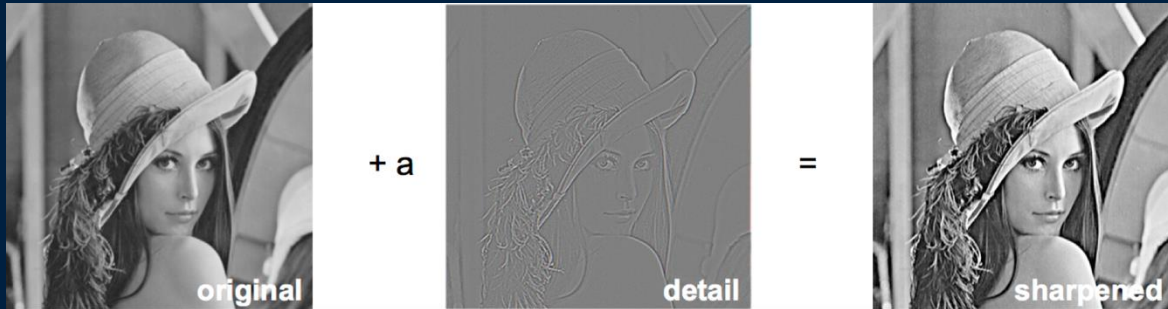


Image Filters

In traditional image processing, image filters must be designed and defined manually for various CV tasks.

Example: sharpening an image by adding image edges to the original image



Convolution Operation

All these filters to detect lines, edges, contours, textures, brightness, etc. have been developed and used manually in traditional image processing tasks! In NN these filters will be developed during **Training** the Network!



Original



Sharpen



Edge Detect



"Strong" Edge
Detect



Feature Extraction

1. In **convolutional neural networks**, we do feature extraction by applying a small set of learnable weights, called a kernel or filter, to local regions of the image.
2. Each filter looks only at a small neighborhood of pixels at a time. This is what we mean by **local features**. Edges, corners, and textures are all local patterns.
3. As the filter slides across the image, the same weights are reused everywhere. This is called **spatial weight sharing**, and it is one of the most important ideas in CNNs.
4. Weight sharing dramatically reduces the number of parameters and allows the network to detect the same feature regardless of its position in the image.
5. We don't use just one filter. We use **multiple filters** in **parallel**, and each one learns to respond to a different visual pattern.
6. One filter might learn horizontal edges, another vertical edges, and others more complex textures.
7. Each filter produces its own **feature map**, and these feature maps are stacked together and passed to the next layer.
8. As we go deeper in the network, features become more abstract — from edges, to shapes, and object parts.

Question

Question: Why are **fully connected neural networks** inefficient for image data?

- A. They cannot use activation functions
- B. They require flattening the image and lose spatial structure
- C. They cannot classify images
- D. They do not support GPUs
- E. They have a very large number of parameters for high-dimensional images

Question

Question: What is the purpose of a **convolution filter (kernel)** in a CNN?

- A. To resize the image
- B. To detect Sharpe images
- C. To compress the dataset
- D. To convert images to grayscale
- E. To detect local patterns such as edges and textures

Question

Question: When a convolution filter slides across an image, what is the output called?

- A. Activation function
- B. Weight matrix
- C. Feature map
- D. Kernel output vector

Objectives

- Understand what deep learning is and why it is effective for computer vision
- Describe the historical milestones of deep learning and CNNs
- Explain the concept of representation learning in deep neural networks
- Understand why fully connected networks are inefficient for images
- Explain how convolution extracts visual features using filters



THE UNIVERSITY OF BRITISH COLUMBIA

